Lab 3 Report

Chen Liangkun 23020036007. Lab group: 4.

Lab date: th November 2025. Report date: 22nd March 2026.

1 Lab Introduction and T,eam Information

In this lab we will use ROS to simulate a drone flying. And also we will read through the ROS documentation.

1.1 Transformations in Practice

1. Assume we have an incoming geometry_msgs::Quaternion quat_msg that holds the pose of our robot. We need to save it in an already defined tf2::Quaternion quat_tf for further calculations. Write one line of C++ code to accomplish this task.

   Ans: According to the doc, This can be done by

```
tf2::fromMsg(quat_msg,quat_tf);
```

◆ fromMsg() [9/19]

void tf2::fromMsg (const geometry_msgs::Quaternion & in,
tf2::Quaternion & out
)

Convert a Quaternion message to its equivalent **tf2** representation. This function is a specialization of the fromMsg template defined in tf2/convert.h.

Parameters

in A Quaternion message type.

out The Quaternion converted to a **tf2** type.

Definition at line **313** of file **tf2_geometry_msgs.h**.

2. Assume we have just estimated our robot's newest rotation and it's saved in a variable called quat_tf of type tf2::Quaternion. Write one line of C++ code to convert it to a geometry_msgs::Quaternion type. Use quat_msg as the name of the new variable.

   Ans: According to the doc we can approach this by

```
quat_msg = toMsg(quat_tf);
```

◆ toMsg() [8/19]

geometry_msgs::Quaternion tf2::toMsg (const tf2::Quaternion & in)

Convert a tf2 Quaternion type to its equivalent geometry_msgs representation. This function is a specialization of the toMsg template defined in tf2/convert.h.

Quaternion

Parameters

in A tf2 Quaternion object.

Returns

The Quaternion converted to a geometry_msgs message type.

Definition at line 297 of file tf2_geometry_msgs.h.

3. If you just want to know the scalar value of a tf2::Quaternion, what member function will you use?

   Ans: The scalar value of the quaternion is usually the w number. So we can do this by

```
scalar_value = quat_tf.getW()
```

4. Assume you have a tf2::Quaternion quat_t. How to extract the yaw component of the rotation with just one function call?

   Ans: According to the doc This can be done by

```
double yaw = tf2::getYaw(quat_t)
```

template<class A >

double tf2::getYaw (const A & a)

Return the yaw of anything that can be converted to a tf2::Quaternion The conventions are the usual ROS ones defined in tf2/LineMath/Matrix3x3.h This function is a specialization of getEulerYPR and is useful for its wide-spread use in navigation

Parameters

a the object to get data from (it represents a rotation/quaternion)

yaw yaw

Definition at line 45 of file utils.h.

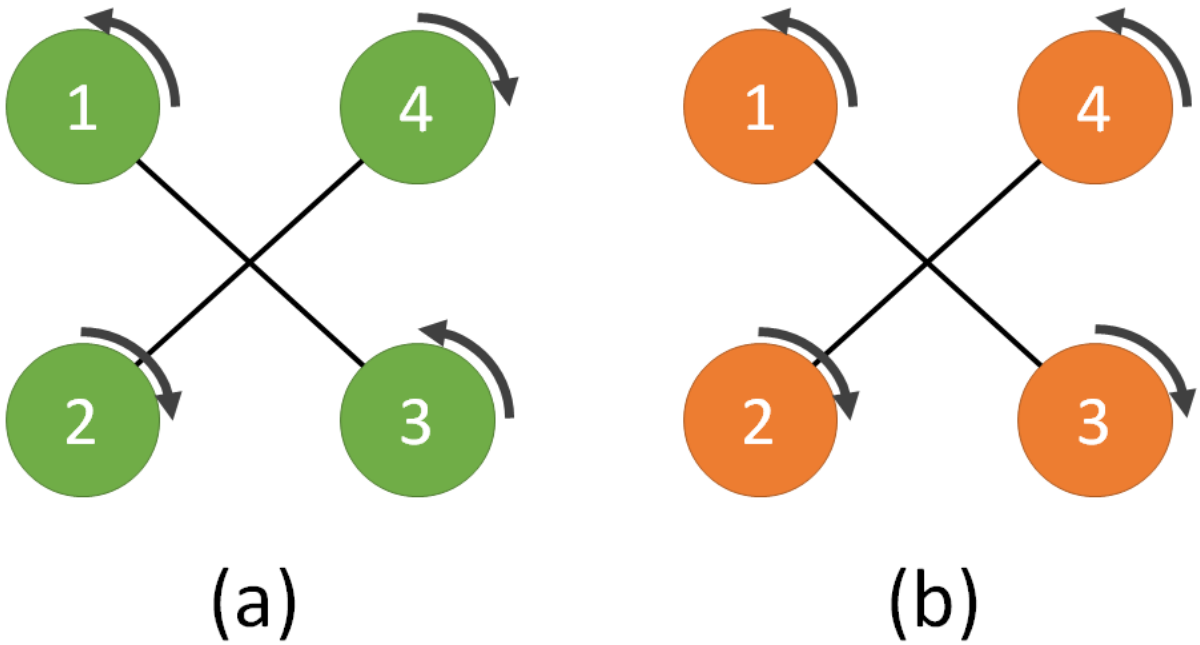
5. Assume you have a geometry_msgs::Quaternion quat_msg. How to you convert it to an Eigen 3-by-3 matrix? Refer to this and this for possible functions. You probably need two function calls for this.

   Ans: Form the doc we can done this by

```
Eigen::Quaterniond q_eigen = tf2::fromMsg(quat_msg);
Eigen::Matrix3d R = q_eigen.toRotationMatrix();
```

void tf2::fromMsg (const geometry_msgs::Quaternion &msg, Eigen::Quaterniond &out)

Convert a Quaternion message type to a Eigen-specific Quaterniond type.
This function is a specialization of the fromMsg template defined in tf2/convert.h.



1.2 Modelling and control of UAVs

1. The figure above depicts two quadrotors (a) and (b). Quadrotor (a) is a fully functional UAV, while for Quadrotor (b) someone changed propellers 3 and 4 and reversed their respective rotation directions.

   Show mathematically that quadrotor (b) is not able to track a trajectory defined in position [x,y,z] and yaw orientation Ψ

   Ans: Let's see the dynamic model of the quadrotor.

$$\begin{bmatrix} m\mathbf{a}^{\mathrm{W}} \\ \mathcal{J}\dot{\boldsymbol{\omega}}^{\mathrm{B}} \end{bmatrix} = \begin{bmatrix} m\mathbf{g}^{\mathrm{W}} \\ -\boldsymbol{\omega}^{\mathrm{B}} \times \mathcal{J}\boldsymbol{\omega}^{\mathrm{B}} \end{bmatrix} + \begin{bmatrix} \mathbf{R}_{\mathrm{B}}^{\mathrm{W}} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} \mathbf{f}_{\mathrm{thrust}}^{\mathrm{B}} \\ \boldsymbol{\tau}_{\mathrm{drag}}^{\mathrm{B}} + \boldsymbol{\tau}_{\mathrm{thrust}}^{\mathrm{B}} \end{bmatrix} = \begin{bmatrix} -mg\mathbf{e}_3 \\ -\boldsymbol{\omega}^{\mathrm{B}} \times \mathcal{J}\boldsymbol{\omega}^{\mathrm{B}} \end{bmatrix} + \begin{bmatrix} \mathbf{R}_{\mathrm{B}}^{\mathrm{W}} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \mathbf{F}\omega \tag{1}$$

   Here, our input is a dimension angular velocity matrix. To get the trajectory we need first multiply them with the matrix $\mathbf{F}$

$$\begin{bmatrix} f_x^{\mathrm{B}} \\ f_y^{\mathrm{B}} \\ f_z^{\mathrm{B}} \\ \tau_x^{\mathrm{B}} \\ \tau_y^{\mathrm{B}} \\ \tau_z^{\mathrm{B}} \end{bmatrix} = \mathbf{F}\mathbf{w} = \begin{bmatrix} \mathbf{c}_f\mathbf{e}_3 & \mathbf{c}_f\mathbf{e}_3 & \mathbf{c}_f\mathbf{e}_3 & \mathbf{c}_f\mathbf{e}_3 \\ c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_1^{\mathrm{B}} \times \mathbf{e}_3 & -c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_2^{\mathrm{B}} \times \mathbf{e}_3 & c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_3^{\mathrm{B}} \times \mathbf{e}_3 & -c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_4^{\mathrm{B}} \times \mathbf{e}_3 \end{bmatrix} \mathbf{w} \tag{2}$$

   Because the quadrotor is symmetries. So x_1 & $x_2 = -x_4$ & $-x_3$ and y_1 & $y_4 = -y_2$ & $-y_3$ Therefore we consider the non-zero part of the matrix

$$F_a = \begin{bmatrix} c_f & c_f & c_f & c_f \\ -c_f y_1^B & -c_f y_1^B & c_f y_1^B & c_f y_1^B \\ -c_f x_1^B & c_f x_1^B & c_f x_1^B & -c_f x_1^B \\ c_d & -c_d & c_d & -c_d \end{bmatrix} \tag{3}$$

We can clearly point out that the rank of F_b is 4, so it can represent the whole input system. However if we consider the F_b

$$F_b = \begin{bmatrix} c_f & c_f & c_f & c_f \\ -c_f y_1^B & -c_f y_1^B & c_f y_1^B & c_f y_1^B \\ -c_f x_1^B & c_f x_1^B & c_f x_1^B & -c_f x_1^B \\ -c_d & c_d & c_d & -c_d \end{bmatrix} \tag{4}$$

Unfortunately, the rank of this matrix is 3 by using elementary transformation method. Thus there will be 1 dimension is compressed. Through the lecture we know to represent a trajectory we need a full rank matrix F. So this proof that t quadrotor (b) is not able to track a trajectory.

2. See the exercise

   Ans: We first simplify the matrix

$$F^b = \begin{pmatrix} 0.1 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.2 \end{pmatrix} (v^b)^2 \tag{5}$$

And as $(v^b)^2 = [|v_x^b|v_x^b, |v_y^b|v_y^b, |v_z^b|v_z^b]^T$

$$F^b = \begin{pmatrix} 0.1|v_x^b|v_x^b \\ 0.1|v_y^b|v_y^b \\ 0.2|v_z^b|v_z^b \end{pmatrix} \tag{6}$$

As the drag force now have include in consideration our dynamic model the part of the force now turns to

$$m\ddot{p} = -mg\mathbf{e_3} + R_B^3(\mathbf{f} + \begin{pmatrix} 0.1|v_x^b|v_x^b \\ 0.1|v_y^b|v_y^b \\ 0.2|v_z^b|v_z^b \end{pmatrix}) \tag{7}$$

Let us recall how we get the force $\mathbf{f}$

$$m\ddot{\mathbf{p}}^w = -mg\mathbf{e_3} - k_p\mathbf{e}_p - k_v\mathbf{e}_v + mg\mathbf{e}_3 + m\ddot{\mathbf{p}}_d^W \iff \tag{8}$$

$$m\ddot{\mathbf{p}}^w = -k_p\mathbf{e}_p - k_v\mathbf{e}_v + m\ddot{\mathbf{p}}_d^W \iff \tag{9}$$

$$(\text{using } \mathbf{e_v} = \dot{\mathbf{e}}_p \quad \ddot{\mathbf{p}}^W - \ddot{\mathbf{p}}_d = \ddot{\mathbf{e}}_p \tag{10}$$

$$m\ddot{\mathbf{e}}_p = -k_p\mathbf{e}_p - k_v\mathbf{e}_v \tag{11}$$

Now we have a new friend F^b, that we need to eliminate it. So we little change the force.

$$f_{ideal}^B = R_w^B(-k_p e_p - k_v e_v + mge_3 + m\ddot{p}_d^w) - \begin{pmatrix} 0.1|v_x^b|v_x^b \\ 0.1|v_y^b|v_y^b \\ 0.2|v_z^b|v_z^b \end{pmatrix} \tag{12}$$

This is because when we substitute the force it will eliminate the F^B now and we can turn our controller back to the most classical geometric controller.

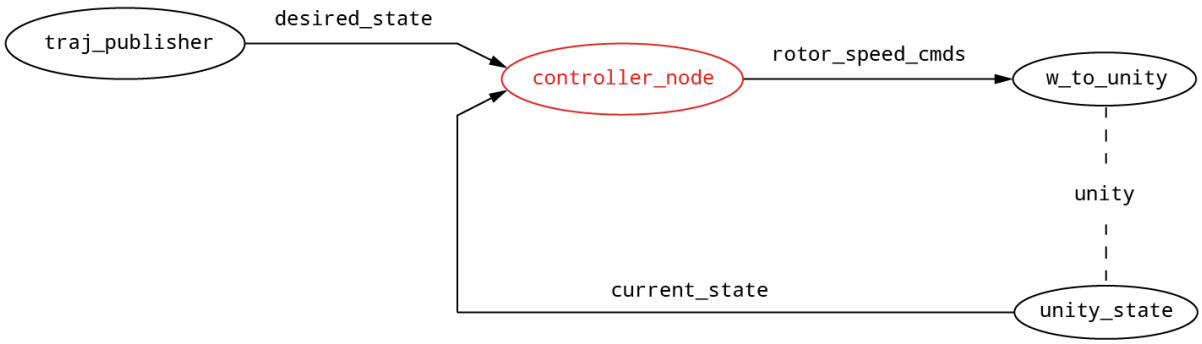


Figure 1: Navigation Stack

2 Team Work

In this section, we implemented a geometric tracking controller on the Special Euclidean Group $SE(3)$ to track a trajectory defined by position, velocity, acceleration, and yaw. The control logic follows the formulation by Lee et al. [1], adapted for the ROS coordinate system conventions.

2.1 communication

We need to communicate with the master and subscribe the current state and desired state. We also need publish the result of the controller. The communication of the controller node is as followed. To achieve this we need to declare the publisher and subscriber and bind them with specific topic. We also need a timer to call the loop.

```
ros::Subscriber cstate_sub;
ros::Subscriber dstate_sub;
ros::Publisher prop_pub;
ros::Timer timer;
cstate_sub = nh.subscribe(current_state, 1000, &controllerNode::onCurrentState,
    this); // subscribe to current state
dstate_sub = nh.subscribe(desired_state, 1000, &controllerNode::onDesiredState,
    this); // subscribe to desired state
prop_pub = nh.advertise<mav_msgs::Actuators>(prop_speeds, 1000);
      // publish propeller speeds
timer = nh.createTimer(ros::Duration(1.0 / hz), &controllerNode::controlLoop,
    this);
```

2.2 subscribe and transfer

Now what we need is to get the desired state and the current state. Let's see the data type of this two state.

```
/desired_state is of type trajectory_msgs/MultiDOFJointTrajectoryPoint and has
    the following fields:
transforms - type geometry_msgs/Transform[]:
```

```
translation: a 3D vector with the desired position of the c.o.m. in the world
    frame
rotation: a quaternion rapresenting a rotation (in our case will be only a yaw
    component)
velocities - type geometry_msgs/Twist[]:
linear: a 3D vector with the desired velocity of c.o.m. in world frame
angular: we will ignore this field as it is not used by the controller
accelerations - type geometry_msgs/Twist[]:
linear: a 3d vector with the desired acceleration of c.o.m. in world frame
angular: we will ignore this field as it is not used by the controller
/current_state is of type nav_msgs/Odometry with the relevant fields:
pose.pose - type geometry_msgs/Pose:
position: a 3d vector with the current position of the c.o.m. in the world
    frame
orientation: a quaternion representing the current orientation of the UAV
twist.twist - type geometry_msgs/Twist:
linear: a 3d vector with the current velocity of c.o.m. in the world frame
angular: a 3d vector with the current angular velocity in the world frame
```

As we use Eigen to calculate the answer we need to do the transfer. Fortunately, ROS has the function to do the job.

Listing 1: desired state

```
tf2::fromMsg(des_state.transforms[0].translation, xd);
tf2::fromMsg(des_state.velocities[0].linear, vd);
tf2::fromMsg(des_state.accelerations[0].linear, ad);
tf2::Quaternion q;
tf2::fromMsg(des_state.transforms[0].rotation, q);
yawd = tf2::getYaw(q);
```

Listing 2: current state

```
tf2::fromMsg(cur_state.pose.pose.position, x);
tf2::fromMsg(cur_state.twist.twist.linear, v);
Eigen::Quaterniond q;
tf2::fromMsg(cur_state.pose.pose.orientation, q);
R = q.toRotationMatrix();
tf2::fromMsg(cur_state.twist.twist.angular, omega);
omega = R.transpose() * omega;
```

2.3 Translational Control

The translational dynamics are controlled by computing the desired total thrust f and the desired direction of the body z-axis, denoted as $\vec{b}_{3_d}$.

First, we compute the position and velocity tracking errors, e_x and e_v:

$$e_x = x - x_d, \quad e_v = v - v_d \tag{13}$$

Using the tracking errors, the desired force vector is computed. Note that unlike the paper which uses a North-East-Down (NED) frame where gravity is positive, our simulation uses

an East-North-Up (ENU) frame where gravity acts in the negative z direction. The desired force vector is calculated as:

$$F_{des} = -k_x e_x - k_v e_v + mg\vec{e}_3 + m\dot{v}_d \tag{14}$$

The total thrust f is the projection of this vector onto the current body z-axis ($\vec{b}_3 = R\vec{e}_3$)[cite: 161]:

Listing 3: Computing Position Errors and Thrust

```
// 5.1 Compute position and velocity errors
ex = xd - x;
ev = vd - v;

// 5.4 Compute the desired wrench (force + torques)
// Note: Gravity term is positive here to compensate for -g in ENU frame
double f = (kx * ex - kv * ev + m * g * e3 + m * ad).dot(R * e3);
```

2.4 Attitude Control

The attitude controller computes the moment M required to align the current rotation matrix R with the desired rotation matrix R_d.

2.4.1 Constructing Desired Attitude R_d

The desired attitude $R_d = [\vec{b}_{1_d}, \vec{b}_{2_d}, \vec{b}_{3_d}]$ is constructed geometrically. The axis $\vec{b}_{3_d}$ aligns with the desired force vector. The heading is determined by projecting the desired yaw direction onto the plane normal to $\vec{b}_{3_d}$:

Listing 4: Construction of Rd

```
// 5.2 Compute the Rd matrix
Eigen::Vector3d b1d;
b1d << cos(yawd), sin(yawd), 0.0;

// Normalized desired force vector
Eigen::Vector3d b3d = (-kx * ex - kv * ev + m * g * e3 + m * ad).normalized();
// Cross product to find orthogonal axes
Eigen::Vector3d b2d = b3d.cross(b1d).normalized();
Eigen::Vector3d proj_b1d = b2d.cross(b3d);

Eigen::Matrix3d Rd;
Rd << proj_b1d, b2d, b3d;
```

2.4.2 Attitude Error and Control Moment

The attitude error e_R is defined on the tangent space of $SO(3)$ using the Vee map $(\cdot)^\vee$[cite: 142]:

$$e_R = \frac{1}{2}(R_d^T R - R^T R_d)^\vee \tag{15}$$

The angular velocity error is simply $e_\Omega = \Omega - R^T R_d \Omega_d$. Since we assume $\Omega_d \approx 0$, this simplifies to $e_\Omega \approx \Omega$[cite: 146]. The control moment M is then:

Listing 5: Attitude Error and Moment Calculation

```
// 5.3 Compute orientation error (er) using Vee map
er = Vee(0.5 * (Rd.transpose() * R - R.transpose() * Rd));
eomega = omega;

// 5.4 Compute Control Moment M
Eigen::Vector3d M = -kr * er - komega * eomega + omega.cross(J * omega);
```

2.5 Motor Mixing and Output

Finally, the computed thrust f and moment M are converted into individual motor velocities. The relationship between the wrench and rotor speeds is linear with respect to the square of the angular velocities. We solve for the squared velocities using the inverse of the allocation matrix $F2W$:

Listing 6: Motor Mixing

```
// 5.5 Recover rotor speeds
Eigen::Vector4d target;
target << f, M(0), M(1), M(2);
Eigen::Vector4d output = F2W.inverse() * target;

// 5.6 Publish control message (using signed_sqrt)
double input1 = signed_sqrt(output(0));
double input2 = signed_sqrt(output(1));
double input3 = signed_sqrt(output(2));
double input4 = signed_sqrt(output(3));
```

This completes the full control loop running at 1000 Hz.

In the end we just publish them and the simulator can subscribe the angular velocity of each rotor and let it fly.

2.6 The Parameter

As we know the mass and the moment of inertia of the UAV. From the experience we can give out the parameter of the k_v, k_x, k_r, k_Ω. Which is 2.5, 3.0, 5.0, 2.0 respectively.

3 Discussion

The lab successfully explored **ROS**-based transformations for converting between **tf2** and **geometry_msgs** types, essential for data handling in the **ROS** ecosystem. The **tf2** :: **fromMsg** and **toMsg** utility functions, along with methods for quaternion data extraction (like **getW**() and **tf2** :: **getYaw**), were applied. Furthermore, a crucial theoretical aspect was established by demonstrating that a quadrotor with reversed propeller

rotation on two opposing rotors (Quadrotor (**b**)) possesses a rank-deficient control allocation matrix $\mathbf{F_b}$, confirming its inability to track trajectories in position and **yaw**. The **SE**(**3**) geometric tracking controller was then implemented, demonstrating effective translational and attitude control with a modified desired force vector to compensate for drag in the **ENU** frame.

This experiment is very interesting. The combination of mathematical derivation and engineering practice is fascinating. Many intuitive and natural engineering implementations are guaranteed by a lot of rigorous and ingenious mathematical thinking. I am a pragmatist. I will feel very uncomfortable if I cannot see the specific production applications of the various knowledge I have learned before. I think they are abstract and difficult to understand. But this experiment solved this problem very well. For example, it allowed me to see how powerful a mathematical structure in discrete mathematics, a group, which is very abstract and whose use is difficult to imagine, is when describing rotation. I think this is also what attracts me very much about this experiment: it allows me to see the significance of learning various knowledge, and I have realized their clever use and powerful ability.

4 Conclusions

This report is reporting the implementation of the Geomeric controller. The process include the former knowledge and the understanding to the paper[1]. First we give out the mathematical process to the whole controller and then we implement it using ROS and Eiegn.

References

[1] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor uav on se(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010. doi: 10.1109/CDC.2010.5717652.